Document Summary

This note introduces the foundational concepts of deep learning, starting with the perceptron as the basic unit of neural networks, followed by essential activation functions that introduce non-linearity. It covers the structure of multi-layer perceptrons (MLPs), optimization techniques like gradient descent, backpropagation for efficient gradient computation, and appropriate loss functions for regression and classification tasks.

Introduction to Deep Learning: Perceptrons, Activation Functions, and Optimization

The Perceptron

Perceptron

*The **perceptron** serves as the simplest building block of neural networks. It processes each input feature by computing a weighted sum and then applies an activation function to produce an output.*

This process begins with the inputs, which are combined linearly using learned weights, and ends with a transformation that introduces non-linearity.

The mathematical formulation of a perceptron is as follows:

Mathematical Expression

$$y = f\left(w_0 + w_1x_1 + w_2x_2 + w_3x_3 + \cdots + w_nx_n\right)$$

In more compact vector notation, this becomes:

Mathematical Expression

$$y = f\left(\sum_{i=0}^{n} w_ix_i\right) = f\left(w^Tx\right)$$

where:

- $x = (x_0, x_1, \ldots, x_n)$ is the input vector, with $x_0 = 1$ serving as the bias term.
- y is the output of the perceptron.
- $f(\cdot)$ denotes a non-linear activation function.
- w_i (including w_0) are the learned weights, where w_0 acts as the bias.

Key Information

Without the activation function $f(\cdot)$, the perceptron closely resembles linear regression. When $f(\cdot)$ is the sigmoid function $\sigma(\cdot)$, it becomes analogous to logistic regression, which is used for binary classification.

The Perceptron in 2D

To build intuition, consider a perceptron in two dimensions, where the input has two features plus the bias. The computation simplifies to:

Mathematical Expression

$$y = f(w_0 x_0 + w_1 x_1 + w_2 x_2)$$

with $x_0 = 1$ (the bias term). By varying the weights w_0, w_1, w_2, the perceptron can represent different linear functions. If the activation is linear, such as $f(x) = x$, the perceptron produces linear decision boundaries, which separate data into two half-planes.

> ***Practical Example***
>
> *Suppose the inputs are $x = (1, 2)$ (bias and one feature), weights are $w = (0.5, -1)$, and the activation is linear. The weighted sum is $0.5 \cdot 1 + (-1) \cdot 2 = -1.5$, so $y = -1.5$. This negative value could indicate one side of a decision boundary.*

For a practical implementation, here is a simple Python example of a perceptron computation using NumPy:

```
import numpy as np

def perceptron(x, w, f):
    """Compute perceptron output: y = f(w^T x)"""
    # x: input vector including bias (x0=1)
    # w: weight vector
    # f: activation function
    weighted_sum = np.dot(w, x)
    return f(weighted_sum)

# Example usage
x = np.array([1.0, 2.0, 3.0])  # Inputs with bias
w = np.array([0.5, 1.0, -0.5])  # Weights
def linear_activation(z):
    return z

y = perceptron(x, w, linear_activation)
print(y)  # Output: 1.0 (0.5*1 + 1*2 + (-0.5)*3 = 1.0)
```

Activation Functions

> ***Activation Functions***
>
> ***Activation functions*** *play a crucial role in neural networks by enforcing specific properties on the outputs (for instance, bounding them like the sigmoid does to [0, 1]) and by introducing non-linearities. These non-linearities are essential for modeling complex patterns, and they also offer benefits such as faster training convergence and sparsity in activations.*

Common activation functions include:

- ReLU (Rectified Linear Unit)
- Sigmoid
- Leaky ReLU
- Tanh (Hyperbolic Tangent)
- Softmax
- Linear (Identity)
- GeLU (Gaussian Error Linear Unit)

To illustrate their differences, the following table summarizes key activation functions, their formulas, and properties:

Activation	Formula	Range	Key Properties
Linear	$f(x) = x$	$(-\infty, \infty)$	No non-linearity; used for regression outputs.
Sigmoid	$\sigma(x) = \frac{1}{1+e^{-x}}$	[0, 1]	Bounds output; smooth but suffers from vanishing gradients.
Tanh	$\tanh(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$	[-1, 1]	Zero-centered; similar to sigmoid but better for some tasks.
ReLU	$f(x) = \max(0, x)$	$[0, \infty)$	Fast computation; introduces sparsity but can cause "dead neurons."
Leaky ReLU	$f(x) = \max(\alpha x, x)$ where $\alpha = 0.01$	$(-\infty, \infty)$	Addresses ReLU's dying neuron issue by allowing small negative slopes.
Softmax	$P_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$	[0, 1] (sums to 1)	Used for multi-class probabilities.
GeLU	$f(x) = x \cdot \Phi(x)$ where Φ is the CDF of standard normal	$(-\infty, \infty)$	Smooth approximation of ReLU; common in modern transformers.

Enforcing Properties on Output (Binary Classification)

In binary classification tasks, the goal is to separate positive (+) and negative (-) samples. Given an input $x \in \mathbb{R}^n$, the model predicts the probability $p(+|x)$, which satisfies $p(+|x) = 1 - p(-|x)$ since the classes are mutually exclusive.

To ensure that $p(+|x)$ lies within [0, 1], we apply an activation function that maps the real line to this interval. The sigmoid function achieves this:

> ***Mathematical Expression***
>
> $$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Here, the raw output of the perceptron (a real number in $\mathbb{R}$) is "squashed" into the probability [0, 1]. As $x \to \infty$, $\sigma(x) \to 1$; as $x \to -\infty$, $\sigma(x) \to 0$; and at $x = 0$, $\sigma(0) = 0.5$.

> ***Practical Example***
>
> *For a logit $z = 2$, $\sigma(2) = \frac{1}{1+e^{-2}} \approx 0.881$, indicating a high probability for the positive class.*

Introducing Non-Linearities

One of the primary reasons for using non-linear activation functions is to prevent the entire network from collapsing into a simple linear transformation. Without non-linearity—for instance, if $f(x) = x$—stacking multiple layers would result in an equivalent single linear operation. Consider two layers:

> ***Mathematical Expression***
>
> $$z = s^T(W_1 x)$$

where s is the output weights of the second layer and W_1 is the first layer's weight matrix. This simplifies to:

> ***Mathematical Expression***
>
> $$z = (s^T W_1)x$$

Implying several issues:

1. The multiple layers reduce to a single effective matrix $W_2 = s^T W_1$.

2. There is no advantage to using multiple layers, as the model remains purely linear.
3. The network cannot capture complex, non-linear relationships in the data.

By introducing non-linear activations between layers, the composition becomes capable of approximating highly complex functions, which is fundamental to deep learning.

Linear Layers and Multi-Layer Perceptrons

Adding Perceptrons: Linear (Fully-Connected) Layer

A single perceptron produces one output, but neural network layers typically generate multiple outputs by combining several perceptrons. This forms a linear (or fully-connected) layer, where every input connects to every output neuron.

For a layer with two output neurons and three inputs (including bias), the outputs are:

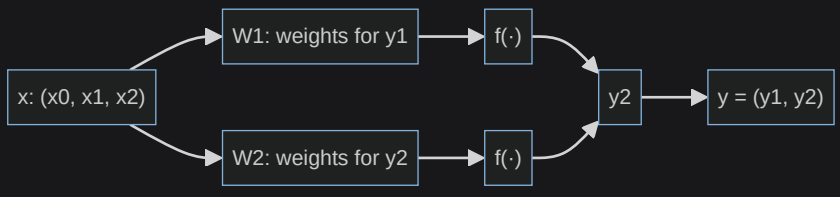
> *Mathematical Expression*
>
> $$y_1 = f(w_1^T x)$$
> $$y_2 = f(q_1^T x)$$
> $$y = \begin{pmatrix} y_1 \\ y_2 \end{pmatrix} = f(W^T x)$$

where the weight matrix is:

> *Mathematical Expression*
>
> $$W = \begin{pmatrix} w_0 & q_0 \\ w_1 & q_1 \\ w_2 & q_2 \end{pmatrix}$$

In this example, there are 6 parameters (weights) in total. The activation f is applied element-wise to the vector $W^T x$.

The following Mermaid diagram illustrates the structure of a simple fully-connected layer with two outputs:



Stacking Layers with Non-Linearities

To increase the model's expressive power, we stack multiple layers, ensuring non-linear activations are applied after each linear transformation (except possibly the output layer). For two layers:

> *Mathematical Expression*
>
> $$z = f(s^T(f(W^T x)))$$

The inner f introduces non-linearity after the first layer, preventing collapse, while the outer f (if present) shapes the final output. This layered composition allows the network to learn hierarchical features, starting from simple patterns in early layers to more abstract ones in deeper layers.

Multi-Layer Perceptron (MLP) Models

> *Multi-Layer Perceptron (MLP)*

Multi-Layer Perceptrons (MLPs) are feedforward neural networks formed by stacking multiple linear layers interspersed with non-linear activations. This design avoids the linearity collapse issue and enables the approximation of complex functions.

The Universal Approximation Theorem provides a theoretical foundation for MLPs:

Mathematical Expression

"For any continuous function g on a compact subset of $\mathbb{R}^d$ and any $\epsilon > 0$, there exists a feedforward network with one

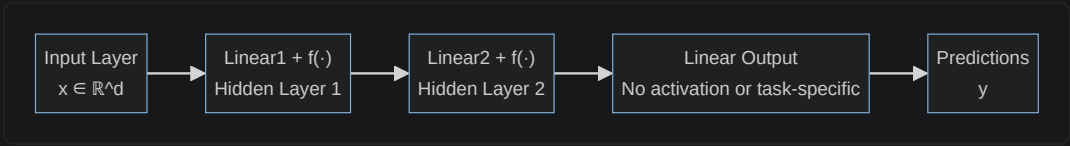
Key insights:

- A single hidden layer is theoretically sufficient for universal approximation.
- However, the theorem does not specify the required number of neurons (width) or layers (depth); in practice, deeper networks with moderate width often perform better by learning hierarchical representations.

A typical MLP structure consists of:

- An input layer that receives the raw features.
- One or more hidden layers, each applying a linear transformation followed by a non-linear activation.
- An output layer that produces the final predictions.

The following Mermaid diagram shows a basic MLP architecture with two layers:



References:

- Cybenko, George. "Approximation by superpositions of a sigmoidal function." Mathematics of control, signals and systems 2.4 (1989): 303-314.
- Hornik, Kurt, Maxwell Stinchcombe, and Halbert White. "Multilayer feedforward networks are universal approximators." Neural networks 2, no. 5 (1989): 359-366.

Activation Functions for Classification Models

In classification models, the layers before the final output produce unnormalized scores called logits. The final activation function then converts these logits into valid probabilities: values in [0, 1] that sum to 1 across classes.

The overall model structure is:

- Input features.
- Repeated blocks of linear transformation followed by non-linear activation (for hidden layers).
- A final linear "head" to produce logits.
- Output activation to get probabilities.
- Final predictions based on probabilities.

Binary Classification

For binary classification, the model predicts the probability of the positive class, $P_{pos}(x)$. The logit z (a scalar real number) from the linear head is passed through the sigmoid:

Mathematical Expression

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

This ensures $P_{pos}(x) \in [0, 1]$. The negative class probability is $P_{neg}(x) = 1 - P_{pos}(x)$, which is also in [0, 1], and the two probabilities sum to 1.

> ***Practical Example***
>
> *If $z = -1$, then $\sigma(-1) \approx 0.269$, so $P_{pos} \approx 0.269$ and $P_{neg} \approx 0.731$.*

Multi-Class Classification

For multi-class classification with n classes labeled $c_1, \ldots, c_n$, the linear head produces a vector of logits $z = (z_1, \ldots, z_n) = model(x)$.

The softmax function converts these to probabilities:

> ***Mathematical Expression***
>
> $$P(c_i|x) = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}}$$

This guarantees that each $P(c_i|x) \in [0, 1]$ and $\sum_i P(c_i|x) = 1$, making it suitable for probabilistic interpretation.

> ***Practical Example***
>
> *Suppose logits are $z = (2, 1, 0)$ for classes Dog, Cat, Bird. Then:*
>
> - $P(Dog) = \frac{e^2}{e^2+e^1+e^0} \approx \frac{7.39}{7.39+2.72+1} \approx 0.721$
> - $P(Cat) \approx 0.265$
> - $P(Bird) \approx 0.014$

For a code implementation of softmax:

```
import numpy as np

def softmax(z):
    """Compute softmax probabilities from logits."""
    exp_z = np.exp(z - np.max(z))  # Subtract max for numerical stability
    return exp_z / np.sum(exp_z)

# Example
logits = np.array([2.0, 1.0, 0.0])
probs = softmax(logits)
print(probs)  # Output: [0.721 0.265 0.014] (approx.)
```

Activation Functions for Regression Models

Regression tasks involve predicting continuous real-valued outputs, such as house prices or temperatures. Unlike classification, there is no need to enforce bounds or normalization on the outputs. Therefore, the output activation is the identity function, $f(x) = x$, also known as linear activation.

This allows the model to produce any real number directly from the final linear layer. Linear activations are primarily used here to maintain the full range of possible predictions.

Defining Weights (Parameters) and Optimization

> ***Parameters and Optimization***

Neural networks learn their weights and biases—collectively referred to as parameters θ—by minimizing a loss function $\mathcal{L}(\theta)$. The loss measures how well the model's predictions match the true targets on the training data. For example, in regression, a common choice is the Mean Squared Error (MSE):

Mathematical Expression

$$\mathcal{L}(\theta) = MSE(\theta) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \theta^T x_i)^2$$

The loss $\mathcal{L}$ is computed over a fixed training dataset (X, y), but we denote it as $\mathcal{L}(\theta)$ to emphasize its dependence on the parameters. The optimization objective is to find the values of θ that minimize this loss.

Linear Regression Optimization

For simple linear regression models, optimization can be solved exactly using a closed-form solution. This involves setting the partial derivative of the loss with respect to the parameters to zero:

Mathematical Expression

$$\frac{\partial \mathcal{L}}{\partial \theta} = 0$$

Because the MSE loss is quadratic in θ, this equation yields an analytical solution, often computed via the normal equation $\theta = (X^T X)^{-1} X^T y$.

As an alternative for intuition (especially useful when thinking about more complex models), one could evaluate $\mathcal{L}(\theta)$ for a grid of possible θ values and select the one yielding the lowest loss. This brute-force approach highlights the search nature of optimization in non-solvable cases.

Optimization for Complex Losses/Models

In deep learning, losses are typically non-convex, and models are too complex for closed-form solutions. Instead, we use iterative optimization algorithms that evaluate the loss repeatedly and update parameters to approach a local minimum.

The process starts with a random initialization: $\theta^{(0)}$. Then, at each iteration t, update the parameters:

Mathematical Expression

$$\theta^{(t+1)} = \theta^{(t)} + \Delta\theta$$

A foundational iterative method is Gradient Descent (GD), which moves the parameters in the direction opposite to the gradient of the loss (the direction of steepest ascent):

Mathematical Expression

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla_\theta \mathcal{L}(\theta^{(t)})$$

In one dimension, this simplifies to:

Mathematical Expression

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \frac{d\mathcal{L}}{d\theta}$$

The hyperparameter α is the learning rate, which controls the size of each step. A small α ensures stability but slows convergence, while a large α speeds up training but risks overshooting the minimum.

Practical Example

Suppose a 1D loss $\mathcal{L}(\theta) = (\theta - 2)^2$, with $\theta^{(0)} = 0$ and $\alpha = 0.1$. The gradient is $\nabla \mathcal{L} = 2(\theta - 2)$, so first update: $\theta^{(1)} = 0 - 0.1 \cdot 2(0 - 2) = 0.4$.

Limitations of Gradient Descent

Despite its effectiveness, vanilla Gradient Descent has several limitations:

- It is highly sensitive to the initial parameter values $\theta^{(0)}$; different starting points can lead to different local minima or saddle points.
- In non-convex landscapes, it may converge to suboptimal local minima rather than the global one.

To mitigate these issues, advanced variants are used, such as:

- Momentum, which accelerates GD in relevant directions and dampens oscillations.
- Adaptive methods like Adam, which adjust learning rates per parameter.
- Learning rate schedules, which decrease α over time for finer convergence.

> ***Key Information***
>
> *If the loss function is convex (e.g., MSE for linear regression), Gradient Descent guarantees convergence to the global minimum from any initialization. However, deep learning losses are rarely convex due to the non-linearities.*

Backpropagation

> ***Backpropagation***
>
> *Efficiently computing the gradient $\nabla_\theta \mathcal{L}$ is critical for optimization in deep networks.* ***Backpropagation*** *achieves this by applying the chain rule of calculus in a backward pass through the model's computational graph, allowing gradients to flow from the output back to the inputs.*

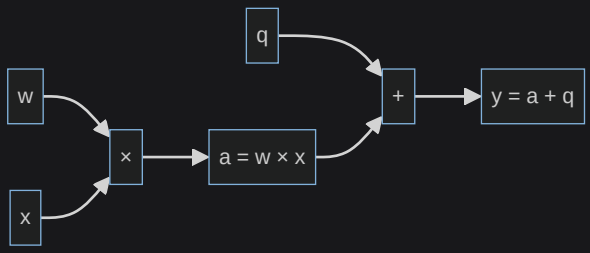
The chain rule underpins this: for a composite function $f(g(x))$, the derivative is $\frac{\partial f}{\partial x} = \frac{\partial f}{\partial g} \cdot \frac{\partial g}{\partial x}$, or $(f \circ g)'(x) = f'(g(x)) \cdot g'(x)$. Backpropagation extends this to entire networks by breaking down computations into small, differentiable steps.

Computational Graph

A computational graph represents the model's forward computation as a directed acyclic graph (DAG). Nodes correspond to operations (e.g., multiplication, addition), and directed edges represent the flow of data (tensors) between them.

> ***Practical Example***
>
> *Consider the simple expression $y = wx + q$, where w, x, q are inputs.*
>
> - *Compute intermediate $a = w \times x$.*
> - *Then $y = a + q$.*
>
> *The graph structure is: Inputs w, x, q flow into a multiplication node to produce a, which then flows into an addition node with q to produce y.*

The following Mermaid diagram visualizes this computational graph:



Backpropagation Example

To see backpropagation in action, consider a dataset with one data point (x, y), a simple model $\hat{y} = \theta_1\theta_2 x$, and squared error loss $\mathcal{L} = (\theta_1\theta_2 x - y)^2$.

Break it into a computational graph:

- $a = \theta_1 \times \theta_2$
- b = a \times x = \theta_ \theta_2 x
- $c = b - y = \theta_1\theta_2 x - y$
- $\mathcal{L} = c^2 = (\theta_1\theta_2 x - y)^2$

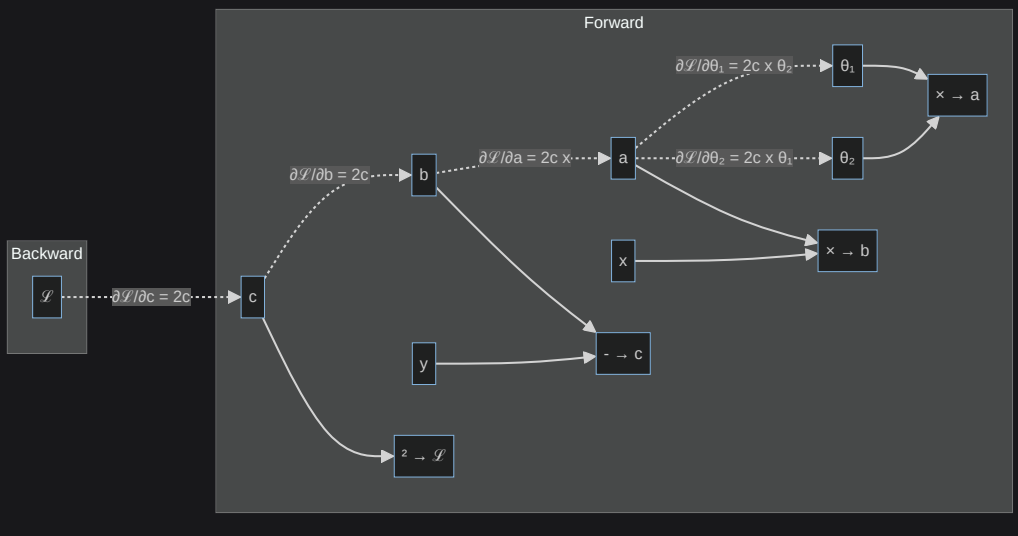
Forward Pass: Starting from inputs θ_1, θ_2, x, y, compute intermediates step-by-step up to $\mathcal{L}$.

Backward Pass: Propagate derivatives from $\mathcal{L}$ back to the parameters using the chain rule:

1. Local gradient at c: $\frac{\partial \mathcal{L}}{\partial c} = 2c$
2. For b: $\frac{\partial \mathcal{L}}{\partial b} = \frac{\partial \mathcal{L}}{\partial c} \cdot \frac{\partial c}{\partial b} = 2c \cdot 1 = 2c$
3. For a: $\frac{\partial \mathcal{L}}{\partial a} = \frac{\partial \mathcal{L}}{\partial b} \cdot \frac{\partial b}{\partial a} = 2c \cdot x = 2cx$
4. For θ_1: $\frac{\partial \mathcal{L}}{\partial \theta_1} = \frac{\partial \mathcal{L}}{\partial a} \cdot \frac{\partial a}{\partial \theta_1} = 2cx \cdot \theta_2 = 2(\theta_1\theta_2 x - y)x\theta_2$
5. For θ_2: $\frac{\partial \mathcal{L}}{\partial \theta_2} = \frac{\partial \mathcal{L}}{\partial a} \cdot \frac{\partial a}{\partial \theta_2} = 2cx \cdot \theta_1 = 2(\theta_1\theta_2 x - y)x\theta_1$

These gradients can then be used in a GD update. In practice, libraries like PyTorch or TensorFlow automate this via autograd.

The following Mermaid diagram outlines the forward and backward passes:



Loss Functions

> ***Loss Functions***
>
> ***Loss functions*** *quantify the discrepancy between predictions and true targets, guiding the optimization process. The choice of loss depends on the task (regression or classification).*

Regression

For regression, losses penalize prediction errors in the output space:

- **Mean Squared Error (MSE):** Emphasizes large errors quadratically.

> *Mathematical Expression*
>
> $$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- **Mean Absolute Error (MAE):** Treats all errors linearly, more robust to outliers.

> *Mathematical Expression*
>
> $$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

> *Practical Example*
>
> *For true values* $y = [3, -1]$ *and predictions* $\hat{y} = [2.5, -0.5]$, *MSE* = $\frac{(3-2.5)^2+(-1-(-0.5))^2}{2} = 0.25$.

Binary Classification

For binary classification, the **Binary Cross-Entropy (BCE)** loss measures the difference between predicted probabilities and binary true labels $y \in \{0, 1\}$, with predictions $\hat{y} \in [0, 1]$ from sigmoid:

> *Mathematical Expression*
>
> $$\mathcal{L} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

- When $y = 1$, it simplifies to $\mathcal{L} = -\log(\hat{y})$.
- When $y = 0$, it simplifies to $\mathcal{L} = -\log(1 - \hat{y})$.

The loss is minimized (close to 0) when $\hat{y} \approx y$, and it increases sharply for mismatches, encouraging confident correct predictions.

For $y = 1$:

- Low loss when $\hat{y} \approx 1$ (e.g., $\hat{y} = 0.99$, $\mathcal{L} \approx 0.01$).
- High loss when $\hat{y} \approx 0$ (e.g., $\hat{y} = 0.01$, $\mathcal{L} \approx 4.6$).

A Python implementation of BCE (per sample):

```
import numpy as np

def binary_cross_entropy(y_true, y_pred):
    """BCE loss for a single sample."""
    y_pred = np.clip(y_pred, 1e-15, 1 - 1e-15)  # Avoid log(0)
    return - (y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))

# Example
y_true = 1.0
y_pred = 0.9
loss = binary_cross_entropy(y_true, y_pred)
print(loss)  # Output: approx. 0.105
```

Multi-Class Classification

For multi-class problems, **Categorical Cross-Entropy** generalizes BCE. With n classes, true labels are one-hot encoded ($y_i = 1$ for the true class i, 0 otherwise), and predictions $\hat{y}_i \in [0, 1]$ come from softmax:

> *Mathematical Expression*

$$\mathcal{L} = -\sum_{i=1}^{n} y_i \log(\hat{y}_i)$$

Since only one $y_i = 1$, this reduces to $\mathcal{L} = -\log(\hat{y}_{true})$ for the true class, penalizing low confidence in the correct class.

> ***Practical Example***
>
> *Suppose logits are $z = (2, 1, 0)$ for classes Dog, Cat, Bird. Then:*
>
> - $P(Dog) = \frac{e^2}{e^2+e^1+e^0} \approx \frac{7.39}{7.39+2.72+1} \approx 0.721$
> - $P(Cat) \approx 0.265$
> - $P(Bird) \approx 0.014$

```
import numpy as np

def softmax(z):
    """Compute softmax probabilities from logits."""
    exp_z = np.exp(z - np.max(z))  # Subtract max for numerical stability
    return exp_z / np.sum(exp_z)

# Example
logits = np.array([2.0, 1.0, 0.0])
probs = softmax(logits)
print(probs)  # Output: [0.721 0.265 0.014] (approx.)
```

References

- Machine Learning
- Neural Networks
- Linear Algebra

- Cybenko, George. "Approximation by superpositions of a sigmoidal function." Mathematics of control, signals and systems 2.4 (1989): 303-314.
- Hornik, Kurt, Maxwell Stinchcombe, and Halbert White. "Multilayer feedforward networks are universal approximators." Neural networks 2, no. 5 (1989): 359-366.